

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 МАМБА-МОДЕЛИ В МАШИННОМ ПЕРЕВОДЕ	4
1.1 Машинный перевод как задача моделирования последовательностей	4
1.2 Механизм внимания и Transformer	5
1.3 Mamba и модели пространства состояний	5
1.4 Mamba только с декодером	7
2 РЕАЛИЗАЦИЯ И АНАЛИЗ ЭФФЕКТИВНОСТИ МОДЕЛЕЙ	9
2.1 Датасет и предобработка данных	9
2.2 Токенизатор	10
2.3 Используемые метрики	11
2.4 Архитектуры моделей	11
2.5 Графики обучения	12
2.6 Итоговое сравнение	14
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17
ПРИЛОЖЕНИЕ А	18

ВВЕДЕНИЕ

Тема данной работы — применение архитектуры Mamba в задаче нейронного машинного перевода. Машинный перевод является одной из классических задач обработки естественного языка. С практической точки зрения она заключается в построении модели, которая по тексту на исходном языке формирует перевод на целевом языке, сохраняя смысл, грамматическую структуру и контекст высказывания.

Долгое время основой нейронного машинного перевода были рекуррентные сети. Такие модели естественным образом работают с последовательностями: они читают токены один за другим и обновляют внутреннее состояние. Однако при увеличении длины предложения рекуррентная обработка становится узким местом, а информация о ранних токенах постепенно теряется. Позже эту проблему во многом решили модели Transformer с механизмом самовнимания [2], но цена такого решения — квадратичная сложность по длине последовательности.

В последние годы активно развиваются модели Mamba, основанные на избирательном обновлении состояния [3]. В отличие от обычного самовнимания, такой блок хранит информацию в скрытом состоянии и обрабатывает последовательность за линейное время. Это делает Mamba интересной для машинного перевода, где модель должна учитывать контекст исходного предложения и уже построенной части перевода.

Отдельный интерес представляет способ построения модели перевода. Классическая схема использует encoder и decoder: encoder обрабатывает исходное предложение, а decoder по его представлениям строит перевод. Другой вариант — модель только с декодером: исходный текст и перевод записываются в одну авторегрессионную последовательность, а модель учится продолжать ее токенами перевода. Такой подход позволяет сравнить Mamba не

только с классической encoder-decoder схемой, но и с более простой единой формулировкой задачи.

Постановка задачи

Целью работы является исследование архитектуры Mamba в задаче нейронного машинного перевода на русско-английском корпусе OPUS-100.

Для достижения поставленной цели необходимо решить следующие задачи:

- рассмотреть encoder-decoder и decoder-only подходы к построению моделей перевода;
- кратко описать принцип работы Mamba и моделей пространства состояний;
- реализовать и обучить модели машинного перевода на основе LSTM, Transformer и Mamba;
- добавить и исследовать вариант Mamba только с декодером;
- построить графики изменения функции потерь и BLEU;
- сравнить модели по качеству перевода и числу параметров.

1. МАМБА-МОДЕЛИ В МАШИННОМ ПЕРЕВОДЕ

1.1. Машинный перевод как задача моделирования последовательностей

Нейронный машинный перевод обычно формулируется как задача условного языкового моделирования. Пусть исходное предложение задано последовательностью токенов $x = (x_1, \dots, x_m)$, а перевод — последовательностью $y = (y_1, \dots, y_n)$. Тогда модель должна оценивать вероятность перевода

$$p(y | x) = \prod_{t=1}^n p(y_t | y_{<t}, x). \quad (1)$$

На практике для такой задачи часто используют архитектуру encoder-decoder. Encoder преобразует исходную последовательность в набор скрытых представлений, а decoder по этим представлениям авторегрессионно строит перевод. В качестве базовой рекуррентной модели в данной работе используется LSTM с механизмом внимания Лионга [1]. Такой выбор позволяет сравнить современные архитектуры с классическим подходом к машинному переводу.

Задачу sequence-to-sequence можно сформулировать и без отдельного encoder. В decoder-only подходе исходное предложение, служебный разделитель и целевой перевод объединяются в одну последовательность. Модель обучается как языковая модель, то есть предсказывает следующий токен по предыдущим токенам, но функция потерь учитывается только на токенах перевода. В этом случае исходное предложение играет роль префикса, который задает условие для генерации. Такая схема используется в современных авторегрессионных языковых моделях и позволяет строить перевод одной моделью без явного разделения на encoder и decoder.

В рекуррентной модели скрытое состояние обновляется

последовательно. В самом общем виде это можно записать так:

$$h_t = f(h_{t-1}, x_t), \quad y_t = g(h_t). \quad (2)$$

Именно наличие состояния делает LSTM и Mamba близкими по общей идее: обе модели накапливают информацию о прошлых токенах. Различие заключается в том, как это состояние параметризуется и насколько эффективно его можно вычислять при обучении.

1.2. Механизм внимания и Transformer

Модели Transformer стали основой многих современных систем машинного перевода [2]. Их ключевой компонент — механизм самовнимания, который позволяет каждому токenu напрямую учитывать другие токены последовательности. Для матриц запросов Q , ключей K и значений V самовнимание записывается следующим образом:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V, \quad (3)$$

где d — размерность векторов ключей.

В encoder-decoder Transformer encoder строит контекстные представления исходного предложения, а decoder использует два вида внимания: причинное самовнимание по уже сгенерированным токенам и внимание к выходам encoder. Такой подход хорошо работает в машинном переводе, но самовнимание требует попарного сравнения токенов. Поэтому вычислительная сложность и расход памяти растут квадратично по длине последовательности. Это ограничение особенно заметно при работе с длинными текстами.

1.3. Mamba и модели пространства состояний

Mamba относится к моделям, которые используют скрытое состояние для обработки последовательности [3]. Базовую идею удобно описать через

дискретную модель пространства состояний:

$$\begin{aligned}h_t &= \bar{A}h_{t-1} + \bar{B}x_t, \\y_t &= Ch_t + Dx_t,\end{aligned}\tag{4}$$

где h_t — скрытое состояние, x_t — входной токен, y_t — выход, а матрицы $\bar{A}$, $\bar{B}$, C и D задают обновление состояния и построение выхода. Если эти параметры фиксированы для всех токенов, модель одинаково обрабатывает все позиции последовательности. Это удобно для вычислений, но ограничивает способность модели избирательно запоминать важные элементы текста.

Mamba решает эту проблему за счет входозависимых параметров. Для каждого токена строятся свои значения Δ_t , B_t и C_t :

$$[\delta_t, B_t, C_t] = W_x x_t + b_x, \quad \Delta_t = \text{softplus}(W_\Delta \delta_t + b_\Delta).\tag{5}$$

После этого состояние обновляется по правилу

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t + D x_t,\tag{6}$$

где $\bar{A}_t$ и $\bar{B}_t$ зависят от Δ_t . Благодаря этому модель может по-разному обрабатывать разные токены: одни сохранять в состоянии, а другие быстро забывать.

Из-за входозависимых параметров Mamba нельзя заранее представить как одну фиксированную свертку. Тем не менее модель остается быстрой благодаря selective scan — параллельному сканированию, которое выполняет рекуррентные обновления за линейное время и эффективно использует современные графические ускорители. Псевдокод блока Mamba приведен в листинге 1.

Листинг 1 — Основные операции блока Mamba

```
1 Input: x : (B, L, D)
2 Parameters: A : (D, N)
3
4 x, z : (B, L, E * D) <- Linear_in(x)
```

```

5 x : (B, L, E * D) <- SiLU(CausalConv1D(x))
6
7 B : (B, L, N) <- s_B(x)
8 C : (B, L, N) <- s_C(x)
9 Delta : (B, L, E * D) <- softplus(Parameter + s_Delta(x))
10
11 A_bar, B_bar <- discretize(Delta, A, B)
12 y : (B, L, E * D) <- selective_scan(A_bar, B_bar, C)(x)
13 y <- y * SiLU(z)
14
15 Output: Linear_out(y)

```

Для машинного перевода это свойство важно, потому что не все токены одинаково значимы для построения перевода. Имена собственные, глаголы и числовые значения часто нужно сохранять точнее, чем служебные слова или отдельные знаки препинания. Поэтому избирательное обновление состояния может быть полезной альтернативой полному самовниманию.

1.4. Mamba только с декодером

В классической схеме encoder-decoder исходное предложение и перевод обрабатываются разными частями модели. В Mamba только с декодером эта граница задается не архитектурой, а форматом последовательности. Исходное предложение, разделитель и перевод объединяются в один авторегрессионный контекст:

$$z = (x_1, \dots, x_m, \text{sep}, y_1, \dots, y_n). \quad (7)$$

Тогда условная вероятность перевода записывается как

$$p(y \mid x) = \prod_{t=1}^n p(y_t \mid x_1, \dots, x_m, \text{sep}, y_{<t}). \quad (8)$$

При обучении модель предсказывает следующий токен для всей объединенной последовательности, но ошибка учитывается только на части, соответствующей переводу:

$$\mathcal{L} = - \sum_{t=1}^n \log p(y_t \mid x_1, \dots, x_m, \text{sep}, y_{<t}). \quad (9)$$

Во время генерации исходное предложение и разделитель подаются как префикс, после чего модель последовательно добавляет токены перевода.

Такой подход хорошо согласуется с причинной природой Mamba: состояние после обработки исходного предложения становится сжатым представлением условия для дальнейшей генерации. Преимущество decoder-only схемы состоит в единой авторегрессионной постановке задачи и более простой структуре модели. Ограничение состоит в том, что связь с исходным предложением проходит через общий последовательный контекст, а не через отдельный слой внимания к выходам encoder.

2. РЕАЛИЗАЦИЯ И АНАЛИЗ ЭФФЕКТИВНОСТИ МОДЕЛЕЙ

В практической части работы были реализованы и обучены четыре модели нейронного машинного перевода: LSTM с механизмом внимания, Transformer, Mamba в схеме encoder-decoder и Mamba только с декодером. LSTM использовалась как базовая рекуррентная модель. Transformer был выбран как стандартная архитектура машинного перевода с механизмом самовнимания. Две модели Mamba позволяют сравнить классическую encoder-decoder схему с decoder-only подходом. Такое сравнение актуально, поскольку Mamba потенциально позволяет работать с длинным контекстом без квадратичной сложности самовнимания [2, 4].

Реализация выполнена на языке Python с использованием библиотеки **PyTorch**. Для всех моделей использовалась одинаковая схема обучения. Модели обучались с оптимизатором AdamW, ограничением нормы градиента и функцией потерь cross-entropy с label smoothing. Планировщик скорости обучения состоял из линейного warmup и последующего косинусного убывания.

Для обучения использовался вычислительный стенд с видеокартой NVIDIA-H100.

2.1. Датасет и предобработка данных

Для обучения использовалась русско-английская часть датасета OPUS-100 [5]. OPUS-100 является многоязычным параллельным корпусом, собранным на основе открытых корпусов OPUS. Русско-английская часть содержит около 1 млн параллельных пар. Каждый пример датасета представляет собой пару предложений: предложение на русском языке и соответствующий ему перевод на английский язык.

В работе использовались стандартные разбиения датасета на

обучающую, валидационную и тестовую части. Во время обучения длина последовательности ограничивалась 256 токенами. Такая граница позволяет отсеять слишком длинные предложения, которые сильно увеличивают расход памяти, но при этом сохраняет большую часть обычных предложений корпуса.

Для дополнительного контроля качества данных применялись две проверки. Во-первых, автоматически проверялся язык предложений. Так выявлялись примеры, где в русской колонке находился не русский текст или наоборот. Во-вторых, для части пар оценивалась семантическая близость предложений. Такая проверка нужна потому, что параллельные корпуса могут содержать технический шум: неполные переводы, перепутанные строки, одинаковые предложения на разных языках.

2.2. Токенизатор

Для токенизации использовалась subword-модель Unigram [6]. В отличие от простого разбиения по словам, subword-токенизация позволяет работать с неизвестными словами, редкими формами и морфологически богатыми языками. Это особенно важно для русского языка, где одно и то же слово может иметь много грамматических форм.

Перед обучением словаря выполнялась нормализация Unicode в форме NFKC. Далее применялась предварительная токенизация:

- пробелы кодировались через metaspace-представление;
- знаки препинания выделялись в отдельные токены;
- числа разбивались на отдельные цифровые токены.

Отдельная обработка чисел и пунктуации важна для перевода. Числа чаще всего должны переноситься в перевод без изменения, а знаки препинания задают структуру предложения. Если такие элементы смешивать с соседними словами, модели сложнее научиться переносить их корректно.

Для encoder-decoder моделей использовались специальные токены начала и конца последовательности, padding-токен и токен неизвестного фрагмента. Для модели только с декодером дополнительно использовался разделитель между исходным предложением и переводом.

2.3. Используемые метрики

Качество машинного перевода нельзя надежно оценивать только функцией потерь, поэтому основной метрикой качества в работе является BLEU. Метрика BLEU сравнивает n -граммы машинного перевода с n -граммами эталонного перевода [7]:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right), \quad (10)$$

где p_n — модифицированная точность по n -граммам, w_n — веса, а BP — штраф за слишком короткий перевод. В работе BLEU вычислялся с помощью библиотеки SacreBLEU [8].

Дополнительно учитывалось число параметров модели. Эта величина важна, потому что две модели с близким качеством перевода могут существенно отличаться по требованиям к памяти и скорости обучения.

2.4. Архитектуры моделей

Первой моделью была LSTM encoder-decoder с механизмом внимания. Encoder является двунаправленным: одна LSTM читает исходное предложение слева направо, вторая — справа налево. Decoder генерирует перевод авторегрессионно. На каждом шаге применяется механизм внимания, который строит контекстный вектор по скрытым состояниям encoder.

Вторая модель построена на архитектуре Transformer. Encoder использует самовнимание для построения представлений исходного предложения. Decoder

применяет причинное самовнимание к уже сгенерированной части перевода и отдельный слой внимания к выходам encoder. Эта модель служит сильной нейросетевой базой для сравнения с Mamba.

Третья модель использует Mamba-блоки в схеме encoder-decoder. В encoder и decoder последовательная обработка выполняется Mamba-блоками, а связь между исходным предложением и переводом задается слоем внимания. Такой вариант сохраняет привычную для машинного перевода структуру encoder-decoder, но заменяет часть последовательной обработки на блоки Mamba.

Четвертая модель — Mamba только с декодером. В этой схеме исходное предложение и перевод рассматриваются как одна авторегрессионная последовательность. Во время обучения модель получает исходный текст и правильный перевод, но ошибка оптимизации считается только по токенам перевода. Во время вывода исходное предложение задает префикс, после чего модель продолжает последовательность токенами перевода.

В таблице 1 приведены основные конфигурации моделей.

Таблица 1 – Основные конфигурации моделей

Модель	Схема	Размер представления	Число слоев	Особенность
LSTM	encoder-decoder	384	4	рекуррентная базовая модель
Transformer	encoder-decoder	768	6+6	самовнимание и слой внимания к encoder
Mamba	encoder-decoder	768	6+6	Mamba-блоки и слой внимания к encoder
Mamba decoder-only	только декодер	512	8	единая авторегрессионная последовательность

2.5. Графики обучения

Все модели обучались на 100 тыс. итераций с размером батча 100. Для BLEU использовались 40 проверок на валидационной выборке.

На рисунке 1 показано изменение функции потерь на обучении. У всех моделей значение функции потерь убывает, что говорит о корректном ходе оптимизации. Transformer и Mamba в схеме encoder-decoder быстрее достигают

меньших значений функции потерь, а LSTM и Mamba только с декодером сходятся более плавно.

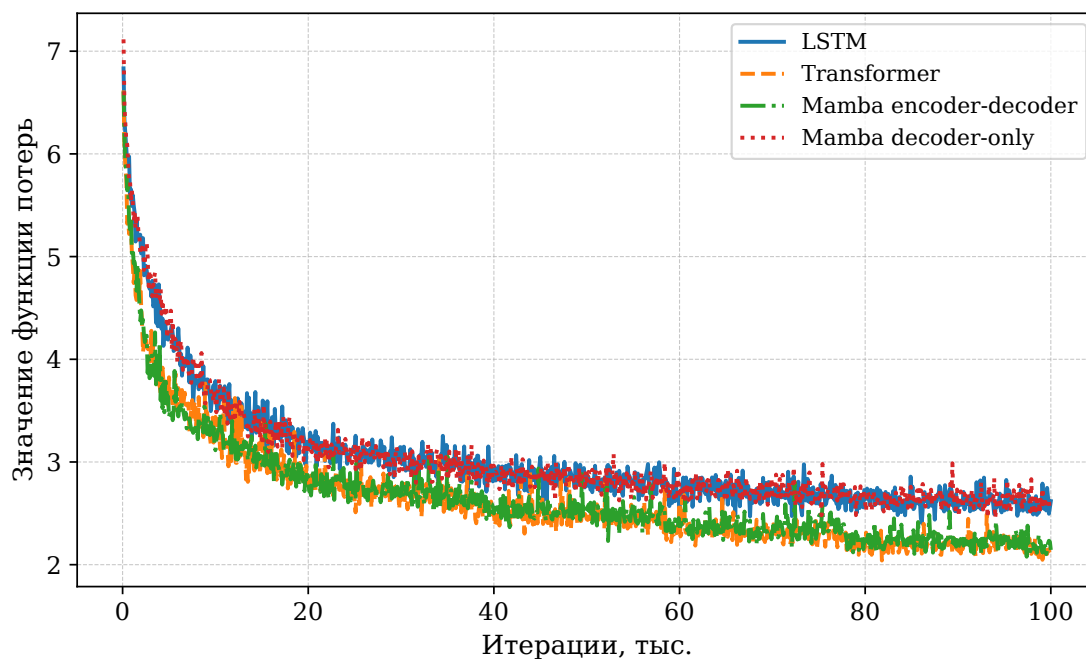


Рисунок 1 – Изменение функции потерь на обучении

На рисунке 2 приведено изменение BLEU на валидационной выборке. По этому графику видно, что Transformer и Mamba encoder-decoder дают близкое качество и заметно превосходят LSTM. Mamba только с декодером набирает качество медленнее и в данном эксперименте уступает encoder-decoder вариантам.

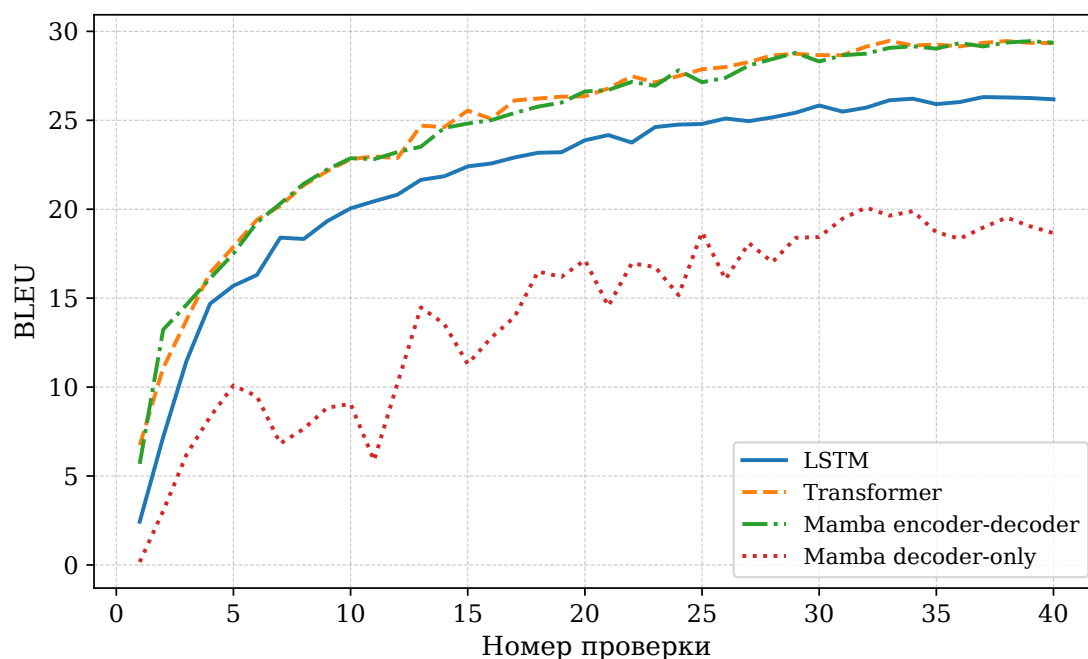


Рисунок 2 – Изменение BLEU на валидационной выборке

2.6. Итоговое сравнение

Итоговые результаты представлены в таблице 2. Для BLEU указано лучшее значение на валидационной выборке. Размер модели указан в миллионах параметров.

Таблица 2 – Итоговое сравнение моделей машинного перевода

Модель	BLEU	Число параметров
LSTM	26.31	73.5M
Transformer	29.47	158.8M
Mamba encoder-decoder	29.45	190.3M
Mamba decoder-only	20.09	90.0M

По результатам видно, что Mamba в схеме encoder-decoder достигает качества, близкого к Transformer, и превосходит LSTM. При этом вариант Mamba только с декодером показал более низкое значение BLEU. Это означает, что для рассматриваемого корпуса явное разделение исходного предложения и

перевода в encoder-decoder схеме оказалось полезным. Тем не менее decoder-only вариант остается важным для сравнения, поскольку он показывает, как Mamba работает в единой авторегрессионной постановке задачи.

ЗАКЛЮЧЕНИЕ

В результате работы была изучена архитектура Mamba применительно к задаче нейронного машинного перевода. Были рассмотрены две постановки sequence-to-sequence задачи: классическая схема encoder-decoder и модель только с декодером, где исходное предложение и перевод образуют одну авторегрессионную последовательность. Также была кратко описана связь Mamba с моделями пространства состояний и идея избирательного обновления скрытого состояния.

В практической части были обучены и сравнены четыре модели: LSTM с механизмом внимания, Transformer, Mamba encoder-decoder и Mamba decoder-only. Основной метрикой качества была BLEU, дополнительно учитывалось число параметров. На русско-английском корпусе OPUS-100 Mamba в схеме encoder-decoder показала качество, близкое к Transformer, и превзошла LSTM.

Модель Mamba только с декодером уступила encoder-decoder вариантам по BLEU. Это показывает, что для данной задачи явное кодирование исходного предложения остается полезным. В то же время decoder-only схема работоспособна и может рассматриваться как отдельное направление для дальнейшего улучшения Mamba-моделей машинного перевода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Luong M.-T., Pham H., Manning C.D. Effective Approaches to Attention-based Neural Machine Translation [Электронный ресурс]. — 2015. — URL: <https://arxiv.org/abs/1508.04025>
2. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser Ł., Polosukhin I. Attention Is All You Need [Электронный ресурс]. — 2017. — URL: <https://arxiv.org/abs/1706.03762>
3. Gu A., Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces [Электронный ресурс]. — 2023. — URL: <https://arxiv.org/abs/2312.00752>
4. Pitorro R., et al. How Effective Are State Space Models for Machine Translation? [Электронный ресурс]. — 2024. — URL: <https://arxiv.org/abs/2402.18637>
5. Zhang B., Williams P., Titov I., Sennrich R. Improving Massively Multilingual Neural Machine Translation and Zero-Shot Translation [Электронный ресурс]. — 2020. — URL: <https://arxiv.org/abs/2004.11867>
6. Kudo T., Richardson J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing [Электронный ресурс]. — 2018. — URL: <https://arxiv.org/abs/1808.06226>
7. Papineni K., Roukos S., Ward T., Zhu W.-J. BLEU: a Method for Automatic Evaluation of Machine Translation [Электронный ресурс] // Proceedings of ACL. — 2002. — URL: <https://aclanthology.org/P02-1040/>
8. Post M. A Call for Clarity in Reporting BLEU Scores [Электронный ресурс] // Proceedings of WMT. — 2018. — URL: <https://aclanthology.org/W18-6319/>

ПРИЛОЖЕНИЕ А

В приложении приведены фрагменты реализации, использованные в работе.

Листинг 2 – Фрагмент реализации LSTM с механизмом внимания

```
1 class LuongAttention(nn.Module):
2     def __init__(self, encoder_dim: int, decoder_dim: int):
3         super().__init__()
4         self.key_proj = nn.Linear(encoder_dim, decoder_dim, bias=False)
5
6     def forward(self, decoder_hidden, encoder_outputs, mask):
7         keys = self.key_proj(encoder_outputs)
8         scores = torch.bmm(keys, decoder_hidden.unsqueeze(2)).squeeze(2)
9         scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)
10        attn_weights = torch.softmax(scores, dim=1)
11        context = torch.bmm(attn_weights.unsqueeze(1), encoder_outputs).squeeze(1)
12        return context, attn_weights
13
14
15 class LuongSeq2Seq(nn.Module):
16     def forward(self, input_ids, output_ids, attention_mask):
17         encoder_outputs, (hidden, cell) = self.encode(input_ids, attention_mask)
18         preds = []
19
20         for t in range(output_ids.size(1)):
21             input_id = output_ids[:, t]
22             logits, hidden, cell = self.decode_step(
23                 input_id,
24                 hidden,
25                 cell,
26                 encoder_outputs,
27                 attention_mask,
28             )
29             preds.append(logits)
30
31         return torch.stack(preds, dim=1)
```

Листинг 3 – Фрагмент реализации блока Mamba

```
1 class MambaMixer(nn.Module):
2     def _selective_scan(self, u, delta, b, c):
3         batch_size, seq_len, d_inner = u.shape
4         a = -torch.exp(self.A_log.float())
5         d = self.D.float()
6         state = u.new_zeros(batch_size, d_inner, self.d_state, dtype=torch.float32)
7         ys = []
8
9         for t in range(seq_len):
10             u_t = u.float()[t, :]
11             delta_t = delta.float()[t, :]
12             b_t = b.float()[t, :]
13             c_t = c.float()[t, :]
14
15             delta_a = torch.exp(delta_t.unsqueeze(-1) * a.unsqueeze(0))
16             delta_b_u = delta_t.unsqueeze(-1) * u_t.unsqueeze(-1) * b_t.unsqueeze(1)
17             state = delta_a * state + delta_b_u
18             y_t = (state * c_t.unsqueeze(1)).sum(dim=-1) + d.unsqueeze(0) * u_t
19             ys.append(y_t)
20
21         return torch.stack(ys, dim=1).to(u.dtype)
22
23     def forward(self, x):
24         xz = self.in_proj(x)
25         x_ssm, z = xz.chunk(2, dim=-1)
26         x_ssm = self._causal_depthwise_conv(x_ssm)
27
28         ssm_params = self.x_proj(x_ssm)
29         delta_raw, b, c = torch.split(
30             ssm_params,
31             [self.dt_rank, self.d_state, self.d_state],
32             dim=-1,
33         )
34         delta = F.softplus(self.dt_proj(delta_raw))
35         y = self._selective_scan(x_ssm, delta, b, c)
36         y = y * F.silu(z)
37         return self.out_proj(y)
```

Листинг 4 – Фрагмент реализации Mamba только с декодером

```
1 class MambaLayer(nn.Module):
2     def forward(self, x: Tensor) -> Tensor:
3         residual = x
4         x = self.norm(x)
5         x = self.mamba(x)
6         return residual + self.dropout(x)
7
8
9 class MambaDecoder(DecoderOnly):
10     def encode(
11         self,
12         input_ids: Tensor,
13         attention_mask: Tensor,
14         type_ids: Tensor,
15     ) -> Tensor:
16         x = self.embedding(input_ids)
17         for layer in self.encoder_layers:
18             x = layer(x, attention_mask, type_ids)
19         return x
20
21     def decode(self, x: Tensor) -> Tensor:
22         for layer in self.decoder_layers:
23             x = layer(x)
24         x = self.norm(x)
25         return self.output(x)
26
27     def forward(
28         self,
29         input_ids: Tensor,
30         attention_mask: Tensor,
31         type_ids: Tensor,
32     ) -> Tensor:
33         x = self.encode(input_ids, attention_mask, type_ids)
34         return self.decode(x)
```
